

Sending YouTube & SoundCloud Channels from Chrome to DJ-CrateBuilder

Date 2026-08-21 Mode Standard research Sources 13 Audience Maintainer

4	49737	Option 1	Blocked
INTEGRATION OPTIONS EVALUATED	EXISTING LOOPBACK PORT REUSABLE AS IPC	RECOMMENDED: DJCRATE:// HANDLER	WEB STORE ROUTE (YOUTUBE POLICY)

Executive Summary

Building a Chrome extension that hands a YouTube or SoundCloud channel URL to DJ-CrateBuilder is clearly feasible, and the app is already better positioned for it than most desktop apps: the single-instance lock in `cratebuilder/singleton.py` holds a loopback TCP socket on port 49737 that already receives messages from second launches [11], and the watchlist has a clean programmatic entry point in `Database.add_watchlist_channel` [12]. Every integration option below reduces to “get the URL from the browser to that socket (or to a fresh launch’s argv).”

Four architectures were evaluated. **Option 1 — a custom `djcrate://` protocol handler plus a thin context-menu extension — is the recommended first build.** It needs roughly 30 lines of registry/argv code in the app, a ~60-line Manifest V3 extension with no host permissions, works when the app is closed (Windows launches it), works from any browser, and even works with no extension at all via a bookmarklet. Its only real cost is Chrome’s one-time “Open DJ-CrateBuilder?” confirmation dialog per site, which the user can permanently allow [8][9].

Option 2 — a native messaging host — is the “correct” Chrome-blessed architecture [1][2]: silent sends, bidirectional replies (e.g. “already in watchlist”), no open network surface. It costs the most: an extra stub executable in the installer, a registry-registered manifest pinned to the extension ID, and Firefox needs a parallel setup. It is the right second phase if the protocol-handler prompt ever becomes annoying.

Option 3 — an HTTP bridge on the existing port — is feasible but currently the riskiest, because Chrome 142 (Oct 2025) began gating requests to loopback addresses behind the new Local Network Access permission, and its interaction with extension-originated fetches is not yet clearly documented [6][7]. It also opens a genuine local attack surface that needs token auth.

One finding constrains distribution, not feasibility: the Chrome Web Store explicitly prohibits extensions that facilitate downloading YouTube content, with rejection, removal, or developer-account bans as enforcement [3][4]. An extension whose sole purpose is feeding a YouTube ripper would very likely be judged against that policy even though it never downloads anything in-browser. The practical answer for a personal tool: load it unpacked in developer mode and never submit it to the store.

1. What the App Already Provides

The research began in the codebase, because the cheapest option is the one that reuses existing machinery.

The singleton socket is a ready-made IPC channel. `acquire_single_instance` binds and listens on `127.0.0.1:49737`; a second launch that loses the bind race calls `request_show`, which connects and sends the ASCII bytes `show`, and the winner's listener thread accepts the connection and restores the window [11]. Two properties matter here: the listener already reads a (16-byte) payload, so extending the protocol to `add <url>` is a small, contained change; and the "second launch forwards to the running instance, then exits" pattern is exactly the relay a protocol handler or native-messaging stub needs. The port is fixed by design, so no discovery step is needed.

The watchlist has a programmatic add path. `Database.add_watchlist_channel(url=..., display_name=..., platform=..., genre=...)` [12] is the same call the UI makes, and the durable channel-link store (`cratebuilder/links.py`) sits beside it. The one wrinkle: an add needs a *genre*, which the browser does not know. The clean resolution is for the app to receive the URL and open its existing add-channel flow pre-filled, letting the user pick the genre in the app rather than in the extension.

Precedent for registry writes exists. `cratebuilder/startup.py` already toggles an HKCU Run key for "start at login," so a per-user protocol registration follows an established in-app pattern and needs no administrator rights.

2. Option 1 — Custom Protocol Handler (`djcrate://`) + Thin Extension

Mechanism. Windows lets any application claim a URI scheme by writing a key under the user's registry classes: `HKCU\Software\Classes\djcrate` with an empty `URL Protocol` value and `shell\open\command` pointing at the exe with `"%1"` [9]. Once registered, any browser that navigates to `djcrate://add?url=https%3A%2F%2Fsoundcloud.com%2Fsomeartist` launches (or signals) the app with the full URI as a command-line argument [8][9]. Browsers show a one-time external-protocol confirmation ("Open DJ-CrateBuilder?") with an *always allow on this site* checkbox.

App-side work

1. A settings toggle (mirroring `startup.py`) that writes/removes the HKCU registry key — pointing at the frozen exe, or at `pythonw.exe <script>` when running from source. 2. Argv parsing at launch: if an argument starts with `djcrate://`, decode it. If this instance wins the singleton bind, process it after the UI is up; if it loses, forward it over the socket instead of plain `show` and exit — the existing second-launch flow with a payload. 3. Widen the singleton listener's read (currently 16 bytes) and dispatch `show` vs `add <url>`, marshalling to the Tk thread as `on_show` already must [11]. 4. On receipt, open the add-channel dialog pre-filled with the URL.

Extension-side work

A Manifest V3 extension with only the `contextMenus` permission: right-click on a page (or a link) on `youtube.com/soundcloud.com` → "Send channel to DJ-CrateBuilder" → the service worker URL-encodes the tab or link URL and navigates to the `djcrate://` URI. No host permissions, no content scripts, no background network access — a minimal, low-maintenance surface. A toolbar button variant is equally trivial.

The bookmarklet bonus

Because the handler is browser-agnostic, a one-line bookmarklet (`javascript:location='djcrate://add?url='+encodeURIComponent(location.href)`) delivers the same feature with zero extension code, in any browser, today. This makes Option 1 incrementally shippable: handler first, extension polish second.

Linux and limitations

The `.deb` can register `x-scheme-handler/djcrate` via its `.desktop` file's `MimeType` entry — the standard freedesktop equivalent — so the same design carries to the Linux channel. The confirmation dialog is per-site (one approval each for `youtube.com` and `soundcloud.com`); URI length is bounded (irrelevant for channel URLs); and

communication is one-way — the extension cannot learn whether the add succeeded. Feedback happens in the app window, which for a “send it over to the app” workflow is arguably where it belongs.

3. Option 2 — Native Messaging Host

Mechanism. Chrome’s native messaging lets an extension exchange JSON messages with a native process over stdio, with each message framed by a 32-bit length prefix [1]. The app installs a small host manifest (name, description, path to the host executable, `"type": "stdio"`, and `allowed_origins` pinned to the extension’s ID) and a registry key `HKCU\Software\Google\Chrome\NativeMessagingHosts\com.syntax.cratebuilder` whose default value is the manifest’s path [1][2]. Chrome launches the host process on demand when the extension calls `chrome.runtime.sendMessage`.

Architecture for CrateBuilder. The host should *not* be the app itself — Chrome would start a second app instance per message. Instead a stub (a small Python script frozen the same way `updater.py` is, or invoked via the system Python) receives `{"action": "add", "url": ...}` on stdin, relays it to the running app over the singleton port, launches the app with the URL as argv when nothing is listening, and writes back `{"ok": true}`. This mirrors the existing app/updater two-executable pattern.

Strengths. After installation there are no prompts of any kind — a click sends silently. The channel is bidirectional, so the extension badge could show “added” or “already watched,” and could later query richer state. Nothing listens on any port for this path, so there is no new network attack surface, and the Local Network Access change is irrelevant to it [1].

Costs and gotchas. It is the most moving parts: a third executable in the installer (host stub), a manifest file, a registry key, and — the classic trap — `allowed_origins` must contain the extension’s exact ID, but an *unpacked* extension’s ID is derived from its folder path unless a `key` field is pinned in `manifest.json`. Forgetting the key pin means the integration breaks when the folder moves. Edge and Firefox each need their own registry location/manifest dialect for the same host [2]. Debugging stdio framing is also famously fiddly compared to `curl`-able HTTP.

Verdict. Feasible and clean, but its advantages over Option 1 (no confirm dialog, feedback in the browser) are quality-of-life rather than capability. Best treated as Phase 2.

4. Option 3 — Localhost HTTP Bridge on Port 49737

Mechanism. Extensions cannot open raw TCP sockets, so speaking to the singleton port directly requires teaching it minimal HTTP: parse a request line, route `POST /add`, reply `200`. The extension then simply `fetches http://127.0.0.1:49737/add` from its service worker, declaring `host_permissions: ["http://127.0.0.1:49737/*"]` [5][13]. One port would serve both single-instance detection and the browser bridge — architecturally tidy, entirely stdlib.

Security obligations. An open loopback HTTP endpoint is reachable by every local process and — absent browser mitigations — by malicious web pages via form posts or DNS rebinding. A production-honest version needs a shared-secret token (generated into the config file, pasted once into the extension’s options page), an `Origin` allowlist, and rejection of non-extension origins. That is all standard, but it is real work and a real ongoing surface, for an app whose current socket protocol is deliberately trivial.

The Local Network Access problem. As of Chrome 142 (28 Oct 2025), Chrome gates requests from public sites to private and loopback addresses behind a new user permission, part of a CSRF-hardening effort [6][7]; enterprise escape hatches exist but are temporary or policy-managed [10]. Critically for this option, Google’s documentation describes the behavior for *websites* and does not clearly specify how extension-originated fetches from a `chrome-extension://` origin are treated [6]. Community signals suggest extensions are prompted or exempted, but this is

unverified — a prototype must be tested against current Chrome before committing. This uncertainty is the main reason not to lead with this option; ironically it also *helps* security by throttling hostile web-page access to the port.

When it wins. If a future roadmap includes other local clients (a phone on the LAN, a Stream Deck plugin, scripts), an authenticated local HTTP API is the most general substrate, and this option becomes strategic rather than merely convenient. It also works when the app is closed only if paired with something that can launch it — which circles back to Options 1 or 2.

5. Option 4 — Baselines Without an Extension

For completeness, three zero-extension paths were assessed. A **clipboard watcher** (app offers “Add copied channel?” when a YouTube/SoundCloud channel URL appears on the clipboard while the window gains focus) requires no browser integration whatsoever and could ship in an afternoon; its downside is discoverability and occasional false positives. The **bookmarklet** rides Option 1’s handler as described. **Drag-and-drop** of a URL onto the Tk window is weak on Windows without the extra `tkinterdnd2` dependency and is not recommended. The clipboard watcher is a genuinely useful complement regardless of which browser option ships.

6. Distribution and Policy Feasibility

This is where “Chrome extension for a YouTube downloader” meets policy reality. The Chrome Web Store’s program policies prohibit extensions that enable or facilitate downloading YouTube content; enforcement ranges from rejection at review to removal to permanent developer-account bans [3][4]. The proposed extension never downloads media in the browser — it forwards a URL — but its listing would necessarily describe feeding a YouTube-ripping desktop app, and the policy targets *facilitation*, not just in-browser downloading [4]. SoundCloud is not covered by that specific rule, but a YouTube-branded listing is a submission risk not worth taking for a personal tool.

The pragmatic path is to never touch the store: load the extension unpacked via Developer Mode (`chrome://extensions` → “Load unpacked”), keep it in the repo (or a private one), and accept Chrome’s mild developer-mode nag on startup. Windows Chrome no longer allows silent sideloading of packed `.crx` files outside enterprise policy, so “unpacked in dev mode” is the realistic personal-install story. This constraint applies equally to all three extension options; notably, Option 1’s bookmarklet fallback is immune to it entirely. Firefox has no equivalent published ban wording aimed at YouTube specifically and allows self-hosted signed add-ons, so a cross-browser future favors options that aren’t Chrome-plumbing-specific — again Option 1.

7. Comparison

Criterion	1 · Protocol handler	2 · Native messaging	3 · HTTP bridge	4 · Clipboard watcher
App-side effort	Small	Medium (stub exe + manifest)	Medium (HTTP + token auth)	Small
Extension effort	Tiny (no host perms)	Small	Small	None
Works when app closed	Yes (OS launches it)	Yes (stub launches it)	No (needs launcher)	Yes
Per-send friction	One-time confirm per site	None	None	Confirm in app
Browser feedback	None	Rich (bidirectional)	Rich	n/a
New attack surface	None meaningful	None (stdio)	Loopback HTTP (token needed)	None

Criterion	1 · Protocol handler	2 · Native messaging	3 · HTTP bridge	4 · Clipboard watcher
Chrome 142 LNA risk	None	None	Unverified [6]	None
Cross-browser	Any browser + bookmarklet	Per-browser manifests	Any (same LNA caveat)	Any
Linux carry-over	.desktop MIMEType	Per-browser manifest dirs	Same code	Same code
Installer changes	Registry toggle (HKCU)	Stub exe + manifest + registry	None	None

8. Recommendation

Phase 1 — ship the `djcrate://` handler with a thin context-menu extension (Option 1), plus the bookmarklet. It is the smallest diff to the app, reuses the singleton socket exactly as designed, uniquely handles the app-not-running case for free, and dodges both the Web Store policy problem (nothing store-worthy exists to reject) and the Local Network Access uncertainty (no network involved). The app-side changes — registry toggle, argv handling, a two-verb socket protocol, pre-filled add dialog — are all unit-testable in `cratebuilder/` without Tk.

Phase 2 (optional) — add a native messaging host (Option 2) if the confirm dialog grates or in-browser feedback (“already in your watchlist”) proves valuable. The host stub relays to the same socket protocol Phase 1 builds, so Phase 1’s work is fully reused.

Defer Option 3 until Chrome’s Local Network Access behavior for extensions is documented or empirically verified, and until there is a second local client that would justify an authenticated HTTP API. **Consider the clipboard watcher (Option 4)** as a cheap, complementary quality-of-life feature independent of the browser work.

9. Limitations & Caveats

This study did not build prototypes; effort estimates come from reading the codebase, not implementation. The LNA-versus-extensions question (Option 3) is explicitly unresolved in Google’s documentation [6] and should be treated as unknown until tested. Chrome Web Store enforcement is described from policy documents and secondary reporting [3][4]; actual review outcomes vary and were not tested. Browser behaviors cited reflect Chrome as of early 2026 and can shift — the LNA rollout itself (Chrome 138 → 142, with policy escape hatches expiring after M152 [10]) shows how quickly loopback rules are moving. Finally, protocol-handler behavior on Linux desktop environments varies by distribution and was assessed from freedesktop conventions, not tested on the `.deb`.

Bibliography

1. Chrome for Developers — *Native messaging*. <https://developer.chrome.com/docs/extensions/develop/concepts/native-messaging>
2. Microsoft Learn — *Native messaging (Microsoft Edge extensions)*. <https://learn.microsoft.com/en-us/microsoft-edge/extensions/developer-guide/native-messaging>
3. Chrome for Developers — *Troubleshooting Chrome Web Store violations*. <https://developer.chrome.com/docs/webstore/troubleshooting>
4. Vidow — *YouTube & Chrome Extensions: why it doesn’t work & what does*. <https://vidow.io/docs/guides/youtube-and-chrome-extensions>
5. MDN Web Docs — *host_permissions (manifest.json)*. https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/manifest.json/host_permissions
6. Chrome for Developers Blog — *New permission prompt for Local Network Access*. <https://developer.chrome.com/blog/local-network-access>
7. Chrome Platform Status — *Local network access restrictions*. <https://chromestatus.com/feature/5152728072060928>
8. Microsoft Edge Blog — *Getting started with Protocol Handlers*. <https://blogs.windows.com/msedgedev/2022/01/20/getting-started-url-protocol-handlers-microsoft-edge/>

9. codestudy.net — *How to register a custom URL protocol in Windows*. <https://www.codestudy.net/blog/how-do-i-register-a-custom-url-protocol-in-windows/>
10. Chrome Enterprise — *LocalNetworkAccessRestrictionsTemporaryOptOut policy*. <https://chromeenterprise.google/policies/local-network-access-restrictions-temporary-opt-out/>
11. DJ-CrateBuilder codebase — `cratebuilder/singleton.py` (single-instance loopback socket, port 49737).
12. DJ-CrateBuilder codebase — `cratebuilder/db.py` (`add_watchlist_channel`, line 871).
13. Chrome for Developers — *Declare permissions*. <https://developer.chrome.com/docs/extensions/develop/concepts/declare-permissions>

Methodology Appendix

Query: Feasibility of a Chrome extension sending YouTube/SoundCloud channel URLs to DJ-CrateBuilder. **Mode:** standard (scope → plan → parallel retrieval → triangulate → synthesize → package).

Evidence base: 5 parallel web searches (native messaging setup, Web Store YouTube policy, MV3 localhost permissions, Local Network Access rollout, Windows protocol-handler registration), one targeted fetch of Google’s LNA announcement, and direct codebase inspection (`singleton.py`, `db.py`, `startup.py` precedent).

Triangulation: the Web Store prohibition and the LNA rollout were each confirmed across three or more independent sources (first-party Google documentation plus secondary reporting). **Key assumption surfaced:** personal-use distribution (unpacked extension) is acceptable — store publication was assessed and advised against rather than assumed. **Known gap:** LNA behavior for extension-originated loopback fetches is undocumented; flagged rather than resolved.